Project Report Format

1. PROJECT TITLE

MedSafe AI: AI-Driven Medical Safety Assistant

2. PROJECT DESCRIPTION

- **System Overview**: An intelligent healthcare assistance platform designed to enhance medicine safety awareness and early risk identification through a unified web-based system.
- **Problem Domain**: Patient safety, medication literacy, and preventative health education.
- **Core Technology Used**: Streamlit, Optical Character Recognition (OCR), Fuzzy Matching algorithms, Curated Medical Databases, and Large Language Models (LLMs).
- **Key Capabilities**: Drug-drug interaction checking, OCR-based prescription extraction, symptom-based guidance, side-effect logging, and emergency risk assessment.
- **Target Users**: Patients, caregivers, and health-tech researchers.
- **Real-world Relevance**: Reduces medication errors caused by illegible prescriptions or unknown interactions without replacing professional medical advice.

3. APPLICATION SCENARIOS / USE CASES

- **End-User Usage (Medication Safety)**: A patient prescribed multiple drugs uses the Interaction Checker to ensure new medications won't cause adverse reactions with existing ones.
- **Caregiver Support (Prescription Understanding)**: A caregiver uploads a photo of a handwritten prescription; the system extracts the active salts and provides educational insights into the medicine's purpose.
- **Educational Usage (Symptom Guidance)**: An individual uses the Symptom & Doubt Solver to understand potential red flags and lifestyle adjustments (like breathing exercises) for minor discomforts.

4. TECHNICAL ARCHITECTURE OVERVIEW

- **Frontend Layer**: Interactive UI built with **Streamlit** for seamless user interaction and image uploads.
- **Backend Layer**: Python-based logic handling **Fuzzy Matching**, rule-based risk scoring, and coordination between AI models.
- **External APIs / Services**: LLM APIs (for safety summaries) and OCR engines (for text extraction).

- **Deployment Environment**: Cloud-hosted (e.g., Streamlit Cloud or AWS) to ensure accessibility.

5. PREREQUISITES

5.1 Software Requirements

- **Development Tools**: Python 3.9+, Git.
- **IDE**: VS Code or PyCharm.
- **Cloud Services**: OpenAI/Gemini API keys for LLM features.

5.2 Libraries / Frameworks / Dependencies

- streamlit, pytesseract (OCR), fuzzywuzzy or thefuzz, pandas, langchain.

5.3 Hardware Requirements

- Standard laptop/PC with at least 8GB RAM for local development and LLM API processing.

6. PRIOR KNOWLEDGE REQUIRED

- **Programming**: Intermediate Python.
- **Frameworks**: Basic Streamlit UI components.
- **AI/ML Fundamentals**: Understanding of Prompt Engineering and OCR workflows.

7. PROJECT OBJECTIVES

- **Technical**: Build a functional OCR and fuzzy matching pipeline for medical data.
- **Performance**: Achieve high accuracy in medicine name extraction from images.
- **Learning Outcomes**: Master the integration of Generative AI with rule-based safety systems.

8. SYSTEM WORKFLOW

1. **User Interaction**: User uploads an image or types medicine names into the Streamlit UI.
2. **Input Handling**: System captures text strings or processes image files.
3. **Processing Logic**: OCR extracts text; Fuzzy Matching aligns text with the curated database.
4. **Core System Execution**: LLM generates safety summaries; Interaction logic checks for conflicts.
5. **Output Generation**: Structured JSON or clear text safety notes are created.
6. **Response Delivery**: Results are displayed on the dashboard with risk-level color coding.

9. MILESTONE 1: REQUIREMENT ANALYSIS & SYSTEM DESIGN

- **Functional Requirements**: OCR extraction, interaction checking, and side-effect logging.
- **Non-Functional Requirements**: Data transparency (educational disclaimer) and low-latency responses.

10. MILESTONE 2: ENVIRONMENT SETUP & INITIAL CONFIGURATION

- **Setup Commands**:
- pip install streamlit pytesseract thefuzz openai

11. MILESTONE 3: CORE SYSTEM DEVELOPMENT

- **Feature 1 – Prescription OCR**: Leverages Generative AI to structure handwritten data into JSON.
- **Feature 2 – Interaction Checker**: Compares input list against a database of known drug conflicts.
- **Feature 3 – Risk Predictor**: Rule-based engine that assigns a "Urgency Level" to symptoms.

12. MILESTONE 4: INTEGRATION & OPTIMIZATION

- **Optimization**: Using fuzzy logic to handle misspellings in medicine names.
- **Security**: Ensuring all guidance includes a "non-diagnostic" disclaimer.

13. MILESTONE 5: TESTING & VALIDATION

Test Case ID	Input	Expected Output	Status
TC-01	Image of "Aspirin"	Correct text extraction & salt identification	Pass

TC-02	Aspirin + Warfarin	Warning: High Interaction Risk	Pass

15. PROJECT STRUCTURE

```
MedSafeAI/
│
├── app.py
├── .env
├── core/
│   ├── ai_engine.py
│   ├── ocr_module.py
│
├── database/jason
├── requirements.txt
```

17. ADVANTAGES & LIMITATIONS

- **Advantages**: High accessibility, reduces patient anxiety, and improves medication literacy.
- **Limitations**: Dependent on image quality; cannot replace professional diagnosis; limited by database scope.